

CS205 C/ C++ Programming - Lab Assignment Template

Name: 殷海洲(Yin Haizhou)

SID: 12210226

Part 1 - Analysis

Implement a calculator that can add, subtract, multiply and divide two numbers. The project can help you become familiar with the steps of C development, basic C syntax, and command-line commands. No GUI is needed for the calculator. A command-line program is fine.

This project can be roughly separated into two parts which are in/output and calculation.

Requirements

1. The programming language should only be C, NOT C++.
2. When you run the program as follows, it will print the expression and the result.
3. It can tell the reason why the operation cannot be carried out.
4. It can tell the reason when the input is not a number
5. Try to resolve calculations with big numbers
6. It could be adapted to another type in which no valid arguments are directly provided.

It is comparatively straightforward to input the equation through command-line according to the requirements, so we directly get our equation from the default arguments of the main function. During this process, we need determine if numbers

or operators are valid, and consequently it is a good idea to construct a function to check before starting to calculate.

It would be rather complicated to deal with calculation since `int` type of data or `double` type is also limited and not precise enough for big numbers. Thus, we may consider not rely much on those datatypes but try to decompose numbers into strings or characters and they can be calculated separately and finally assembled. *(I will also attach my attempt to build up my own theory of calculation)*

Also, We can choose to rely on the funds of previous people who have tried to build their theory of calculation. We may use them as a reference to solve problems. And we use it by `#include <gmp.h>`

Part 2 - Code

Preprocessing

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <stdlib.h>
#include <gmp.h>
```

Define a function to determine type of numbers

In total, there are three types --- invalid, ordinary, and scientific numbers.

To exclude invalid inputs, we need to check each character and its arrangement of them.

```

int Istype(char *num)
{
    int negative = 0; //negative marks
    int expo = 0; //exponetial denotation
    int decimal = 0; //decimal dot
    int type = 1; //default valid number
    for (int i = 0; i <= strlen(num)-1; i++)
    {
        if(num[i] < '0' || num[i] > '9')
        {
            if (num[i] == '+' || num[i] == '-')
            {
                if(i!=0)
                    type = 0;
            }
            else if ((num[i] == 'e' || num[i] == 'E') && i != 0)
            {
                if(expo == 0)
                {
                    expo++;
                    type = 2;
                    if (num[i+1] != '+' && num[i+1] != '-' && (num[i+1] < '0' || num[i+1] > '9'))
                        type = 0;
                    i++;
                    decimal = 0;
                    //if "e" appear, decimal dot clear
                }
                else
                    type = 0;
            }
            else if (num[i] == '.')
            {
                if(decimal == 0)
                {
                    decimal++;
                    type = 2;
                }
                else type = 0;
            }
            else

```

```

        type = 0;
    }
}
return type;
}
//Determine type of a number

```

Define a function to calculate and return a integer or decimal.

We take advantage of gmp package which includes a special datatype of mpf_t with sufficient length.

It's important to follow the grammar of the package to calculate the result while avoiding unnecessary trouble. It is also essential to consider how to get a decent result without redundant affix.

```

void Excal(char *num1, char *num2, char *operator)
{
    mpf_t a,b,result;
    mpf_init(a);
    mpf_init(b);
    mpf_init(result);
    mpf_set_str(a,num1,10);
    mpf_set_str(b,num2,10);
    int validation = 1;
    if(strlen(operator) != 1)
    {
        printf("Invalid Input. Please check your operator.\n");
    }
    else
    {
        if (strcmp(operator,"+") == 0)
        {
            mpf_add(result, a, b);

```

```

}
else if (strcmp(operator, "-") == 0)
{
    mpf_sub(result, a, b);
}
else if (strcmp(operator, "*") == 0)
{
    mpf_mul(result, a, b);
}
else if (strcmp(operator, "/") == 0)
{
    if (mpf_cmp_ui(b, 0) == 0)
    {
        printf("A number cannot be divided by zero.\n");
        validation = 0;
    }
    else
        mpf_div(result, a, b);
}
else
{
    printf("Invalid Input\n");
    validation = 0;
}

//kill zeros(disorder)
int precision = mpf_get_default_prec();
char *numberstring;
gmp_asprintf(&numberstring, "%.*Ff", precision, result);

if (numberstring != NULL && validation != 0)
{
    int len = strlen(numberstring);
    char *dot = strchr(numberstring, '.');

    if (dot != NULL) {
        for (int i = len - 1; i >= 0; i--)
        {
            if (numberstring[i] == '0' || numberstring[i] == '.')
                numberstring[i] = '\0';
            else

```

```

        break;
    }
}

printf("%s %s %s = ", num1, operator, num2);
printf("%s\n", numberstring);
free(numberstring);
}
}
mpf_clear(a);
mpf_clear(b);
mpf_clear(result);
}
//Versatile Calculator

```

Scientific notation is a special type of number. To deal with it, we not only offer a type but also define a function to give results of scientific numbers. This function resembles the one above, but they trouble in different aspects, which I would like to show later.

```

void SciExcal(char *num1, char *num2, char *operator)
{
    mpf_t a,b,result;
    mpf_init(a);
    mpf_init(b);
    mpf_init(result);
    mpf_set_str(a,num1,10);
    mpf_set_str(b,num2,10);
    mpf_set_default_prec(400);

    if(strlen(operator) != 1)
    {
        printf("Invalid Input. Please check your operator.\n");
    }
    else
    {

```

```

    if (strcmp(operator, "+") == 0)
    {
        mpf_add(result, a, b);
    }
    else if (strcmp(operator, "-") == 0)
    {
        mpf_sub(result, a, b);
    }
    else if (strcmp(operator, "*") == 0)
    {
        mpf_mul(result, a, b);
    }
    else if (strcmp(operator, "/") == 0)
    {
        if (mpf_cmp_ui(b, 0) == 0)
            printf("A number cannot be divided by zero.\n");
        else
            mpf_div(result, a, b);
    }
    else
        printf("Invalid Input\n");

    mpf_t exp;
    mpf_init_set_d(exp, 0);
    while (mpf_cmp_d(result, 10) >= 0 || mpf_cmp_d(result, -10) <= 0)
    {
        mpf_div_ui(result, result, 10);
        mpf_add_ui(exp, exp, 1);
    }
    if (mpf_cmp_d(result, 10) >= 0)
    {
        mpf_div_ui(result, result, 10);
        mpf_add_ui(exp, exp, 1);
    }
    printf("%s %s %s = ", num1, operator, num2);
    gmp_printf("%.Ffe%.Ff\n", result, exp);
    mpf_clear(exp);
}
mpf_clear(a);
mpf_clear(b);
mpf_clear(result);

```

```
//scientific notation
```

Part 3 - Result & Verification

Test case 1

```
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator 2 + 3  
2 + 3 = 5  
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator 2 - 3  
2 - 3 = -1  
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator 2 '*' 3  
2 * 3 = 6  
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator 2 / 3  
2 / 3 = 0.6666666666666667  
○ chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$
```

1.jpg

Test case 2

```
chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator 3.14 / 0
A number cannot be divided by zero.
chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$
```

2.jpg

It tells why operation fails.

Test case 3

```
chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$ ./calculator a \* 2
The input cannot be interpret as numbers!
chlorine@DESKTOP-CR08L6M:/mnt/e/C++/Calculator$
```

3.jpg

It tells when not a number.

Test case 4


```
● chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$ ./calculator 987654321 \* 987654321
987654321 * 987654321 = 975461057789971041
○ chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$
```

4.jpg

It can process large numbers.

Test case 5

```
● chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$ ./calculator 1e200 \* 1e200
1e200 * 1e200 = 1e400
○ chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$
```

5.jpg

Even scientific numbers.

Test case 6

```
● chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$ ./calculator
2 + 3
2 + 3 = 5
2 * 3
2 * 3 = 6
quit
○ chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$
```

6.jpg

Test case 7

```
quit
● chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$ ./calculator 1 + 1 + 1 + 1
Invalid Input. Please type in only two numbers and an operator like 1 + 1
○ chlorine@DESKTOP-CRO8L6M:/mnt/e/C++/Calculator$
```

7.jpg

It reminds users how to use it.

Part 4 - Difficulties and Solution

Due to that I had little thing familiar with C/C++ since I am beginner of C programming, I indeed used GPT to assist to enlarge the range and my sight to the great programming world and to provide me with necessary functions and methods, though this work is based on my structure and thoughts.

Main problems

Killing zeros is a very important step in our code since all floats are set with a default precision of 6. Then, we first use `gmp_asprintf` to convert the gmp type float into a string so that we could cut off zeros from back. It needs to be careful that under which circumstances to kill zeros.

I happen to find that standard output of scientific notation, which is supposed to be aeb, in which a is a number between 1 and 10, becomes aeb in which a is smaller than 1. In order to fix this, I repeatedly turns numbers into strings to fix, and combine them to be (10a)e(b-1).

Unfortunately, I encounter `segmentation fault` for several times when using `mpf_get_str` to extract what I want into strings. It turns out that be away from those unfamiliar methods that GPT offers.

Appendix

In this section, I would like to show my initial attempt on this project just as what I have done in EE205.

Calculation

When I first got down to this project, I refused to use any outer source package but I failed two weeks later since I had spent almost a whole day to code addition while I was not familiar with operations that could be done on all kinds of datatype.

```
void add(char num1[max],char num2[max],char result[max+1])
{
```

```

int carry = 0;

for (int i = max-2; i >= 0; i--)
{
    int digit1 = (num1[i]);
    int digit2 = (num2[i]);

    int sum = digit1 + digit2 + carry;
    if (sum == 0)
    ;
    else if(sum >= 97 && sum <= 106)
    {
        result[i+1] = sum - 48;
        carry = (sum - 96) / 10;
    }
    else
    {
        result[i+1] = sum - 58;
        carry = (sum - 96) / 10;
    }
}
if(carry != 0)
    result[0] = carry + '0';
else
    for (int i = 1; i <= 198; i++)
        result[i-1] = result[i];
result[max + 1] = '\0';
}

//define a function for addition

```

(Obviously, it was not tidy nor efficient. I had even used ASCII to calculate)

This addition was based on an idea of character calculation in two strings and failed because I couldn't adapt it to fit decimals as well.

The second try was on exponential numbers when I tried to figure out multiplication between $1e200$ and $1e200$.

```

void decompose(const char *input, char **beforeE, char **afterE)
{
    const char *posi = strchr(input, 'e');
    if (posi == NULL)
    {
        posi = strchr(input, 'E');
    }
    size_t eIndex = posi - input;

    *beforeE = malloc(eIndex + 1);
    strncpy(*beforeE, input, eIndex);
    (*beforeE)[eIndex] = '\0';

    *afterE = strdup(posi + 1);
}
//指数拆解函数

char* combine(double num1, double num2)
{
    char *result = malloc(100);

    sprintf(result, "%g", num1);
    strcat(result, "e");
    char temp[10];
    sprintf(temp, "%g", num2);
    strcat(result, temp);

    return result;
}
//指数合成函数

```

I wrote two functions to decompose and to combine numbers to a single exponential number. Thus, I could calculate any type of exponential number with base equal to 10.

However, despite success in exponential counting, I cannot solve $987654321 * 987654321$ using `double` because it was not precise enough.

Eventually, to make sure my precision to convince my boss my teacher, and my instructor, I choose to include `gmp.h` to realize sufficiently accurate calculation.

Neglectable experience and comments

not important at all

In the end, I feel like expressing my enthusiasm in exploring C/C++ programming. Project 1 indeed cost me a large amount of time while I learned a lot in programming. I'm looking forward to that my ability in programming will be further and further strengthened so that finally I will have managed to accomplish most tasks and challenges.